Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра интеллектуальных информационных технологий

Лабораторная работа №4

По дисциплине «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием RBM»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Данилюк Т.В.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью RBM

Общее задание

1. Взять за основу нейронную сеть из лабораторной работы №3. Выполнить обучение с предобучением, используя стек ограниченных машин Больцмана (RBM – Restricted Boltzmann Machine), алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев как RBM выбрать самостоятельно.
2. Сравнить результаты, полученные при
- обучении без предобучения (ЛР 3);
- обучении с предобучением, используя автоэнкодерный подход (ЛР3);
- обучении с предобучением, используя RBM.
3. Обучить модели на данных из ЛР 2, сравнить результаты по схеме из пункта 2;
4. Сделать выводы, оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№	Выборка	Тип задачи	Целевая переменная
2	https://archive.ics.uci.edu/dataset/189/parkinsons+telemonitoring	регрессия	motor_UPDRS

Ход работы

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.metrics import accuracy_score, f1_score, mean_absolute_error, mean_squared_error,
r2_score
import urllib.request
import io

# --- Параметры ---
HIDDEN = [64, 32, 16]
AE_EPOCHS = 30
RBM_EPOCHS = 30
FT_EPOCHS = 60
BATCH = 64
LR_AE = 1e-3
LR_FT = 1e-3
LR_RBM = 0.01
LR_GRBM = 0.002
SEEDS = [0, 1, 2, 3, 4]
MODES = ["без предобучения", "автоэнкодер", "RBM"]
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"Используемое устройство: {device}")


# --- Загрузка данных ---
def load_parkinsons():
    print("Загрузка Parkinsons Telemonitoring...")
    url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/parkinsons/telemonitoring/parkinsons_updrs.data"
    req = urllib.request.urlopen(url)
    df = pd.read_csv(io.StringIO(req.read().decode('utf-8')))

    y = df["motor_UPDRS"].values.astype(np.float32)
    X = df.drop(columns=["subject#", "motor_UPDRS", "total_UPDRS", "test_time", "age",
"sex"]).values.astype(np.float32)
    return X, y, "regression"


def load_mushroom():
    print("Загрузка Mushroom...")
```

```
    url = "https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.data"
    req = urllib.request.urlopen(url)
    data = req.read().decode('utf-8')
    columns = [
        "poisonous", "cap-shape", "cap-surface", "cap-color", "bruises", "odor",
        "gill-attachment", "gill-spacing", "gill-size", "gill-color",
        "stalk-shape", "stalk-root", "stalk-surface-above-ring",
        "stalk-surface-below-ring", "stalk-color-above-ring",
        "stalk-color-below-ring", "veil-type", "veil-color", "ring-number",
        "ring-type", "spore-print-color", "population", "habitat"
    ]
    df = pd.read_csv(io.StringIO(data), names=columns).replace("?", np.nan).dropna()
    y = df["poisonous"].values
    X = df.drop(columns=["poisonous"])
    return X, y, "classification"


# --- Подготовка данных ---
def prepare(X, y, kind, seed):
    if kind == "classification":
        y = LabelEncoder().fit_transform(y)

    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.2, random_state=seed, stratify=(y if kind == "classification" else None))

    if kind == "regression":
        sc = StandardScaler().fit(X_tr)
        X_tr, X_te = sc.transform(X_tr), sc.transform(X_te)
        y_tr = y_tr.reshape(-1, 1)
        y_te = y_te.reshape(-1, 1)
    else:
        enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit(X_tr)
        X_tr, X_te = enc.transform(X_tr), enc.transform(X_te)

    return (X_tr.astype(np.float32), X_te.astype(np.float32),
            y_tr.astype(np.float32 if kind == "regression" else np.int64),
            y_te.astype(np.float32 if kind == "regression" else np.int64))


# --- Построение модели ---
def build_mlp(in_dim, out_dim, kind):
    layers, prev = [], in_dim
    for h in HIDDEN:
        layers += [nn.Linear(prev, h), nn.ReLU()]
        prev = h
    layers.append(nn.Linear(prev, 1 if kind == "regression" else out_dim))
    return nn.Sequential(*layers)


# --- Обучение модели ---
def train_model(model, X_tr, y_tr, X_te, y_te, kind, seed):
    torch.manual_seed(seed)
    model.to(device)
    opt = optim.Adam(model.parameters(), lr=LR_FT)

    # Выбор функции потерь
    crit = nn.MSELoss() if kind == "regression" else nn.CrossEntropyLoss()

    loader = DataLoader(TensorDataset(torch.tensor(X_tr), torch.tensor(y_tr)),
                        batch_size=BATCH, shuffle=True)
    Xte, yte = torch.tensor(X_te).to(device), torch.tensor(y_te).to(device)
    hist = {"train_loss": [], "test_loss": []}

    for _ in range(FT_EPOCHS):
        model.train()
        tot = 0.0
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            opt.zero_grad()
            pred = model(xb)
            loss = crit(pred, yb)
            loss.backward()
            opt.step()
            tot += loss.item() * len(xb)
        hist["train_loss"].append(tot / len(X_tr))

        model.eval()
```

```
            with torch.no_grad():
                hist["test_loss"].append(crit(model(Xte), yte).item())

    model.eval()
    with torch.no_grad():
        pred = model(Xte).cpu().numpy()

    if kind == "regression":
        return pred, hist
    else:
        return pred.argmax(1), hist


# --- Вспомогательные функции для предобучения ---
def linear_layers_of(model):
    return [m for m in model if isinstance(m, nn.Linear)][:-1]


def pretrain_autoencoder(model, X_tr, seed):
    torch.manual_seed(seed)
    H = torch.tensor(X_tr).to(device)
    for layer in linear_layers_of(model):
        enc = nn.Sequential(nn.Linear(layer.in_features, layer.out_features), nn.ReLU()).to(device)
        dec = nn.Linear(layer.out_features, layer.in_features).to(device)
        opt = optim.Adam(list(enc.parameters()) + list(dec.parameters()), lr=LR_AE)
        crit = nn.MSELoss()
        loader = DataLoader(TensorDataset(H), batch_size=BATCH, shuffle=True)
        for _ in range(AE_EPOCHS):
            for (xb,) in loader:
                opt.zero_grad()
                crit(dec(enc(xb)), xb).backward()
                opt.step()
        layer.weight.data.copy_(enc[0].weight.data)
        layer.bias.data.copy_(enc[0].bias.data)
        with torch.no_grad():
            H = enc(H)


class RBM:
    def __init__(self, n_vis, n_hid, gaussian_visible=False, lr=0.01):
        self.W = (0.01 * torch.randn(n_vis, n_hid)).to(device)
        self.b = torch.zeros(n_vis, device=device)
        self.c = torch.zeros(n_hid, device=device)
        self.gaussian = gaussian_visible
        self.lr = lr

    def hidden_prob(self, v):
        return torch.sigmoid(v @ self.W + self.c)

    def visible_mean(self, h):
        a = h @ self.W.T + self.b
        return a if self.gaussian else torch.sigmoid(a)

    def fit(self, V, epochs, batch=BATCH):
        loader = DataLoader(TensorDataset(V), batch_size=batch, shuffle=True)
        errors = []
        for _ in range(epochs):
            tot = 0.0
            for (v0,) in loader:
                ph0 = self.hidden_prob(v0)
                h0 = torch.bernoulli(ph0)
                v1 = self.visible_mean(h0)
                ph1 = self.hidden_prob(v1)
                n = v0.shape[0]
                self.W += self.lr * (v0.T @ ph0 - v1.T @ ph1) / n
                self.b += self.lr * (v0 - v1).mean(0)
                self.c += self.lr * (ph0 - ph1).mean(0)
                tot += ((v0 - v1) ** 2).mean().item() * n
            errors.append(tot / len(V))
        return errors


def _is_binary(X):
    return bool(np.isin(X, [0.0, 1.0]).all())


def pretrain_rbm(model, X_tr, seed):
```

```
    torch.manual_seed(seed)
    V = torch.tensor(X_tr).to(device)
    errs_all = []
    for k, layer in enumerate(linear_layers_of(model)):
        gauss = (k == 0) and not _is_binary(X_tr)
        rbm = RBM(layer.in_features, layer.out_features,
                  gaussian_visible=gauss, lr=LR_GRBM if gauss else LR_RBM)
        errs_all.append(rbm.fit(V, RBM_EPOCHS))
        layer.weight.data.copy_(rbm.W.T)
        layer.bias.data.copy_(rbm.c)
        with torch.no_grad():
            V = rbm.hidden_prob(V)
    return errs_all


# --- Главный цикл эксперимента ---
def run_dataset(name, X, y, kind):
    print(f"\n{'=' * 20} {name} ({kind}) {'=' * 20}")
    rows, last = [], {}

    for seed in SEEDS:
        X_tr, X_te, y_tr, y_te = prepare(X, y, kind, seed)
        out_dim = 1 if kind == "regression" else len(np.unique(y_tr))

        for mode in MODES:
            torch.manual_seed(seed)
            model = build_mlp(X_tr.shape[1], out_dim, kind)
            pre = None

            if mode == "автоэнкодер":
                pretrain_autoencoder(model, X_tr, seed)
            elif mode == "RBM":
                pre = pretrain_rbm(model, X_tr, seed)

            pred, hist = train_model(model, X_tr, y_tr, X_te, y_te, kind, seed)

            row = {"seed": seed, "mode": mode}
            if kind == "regression":
                row["MAE"] = mean_absolute_error(y_te, pred)
                row["RMSE"] = np.sqrt(mean_squared_error(y_te, pred))
                row["R2"] = r2_score(y_te, pred)
            else:
                row["accuracy"] = accuracy_score(y_te, pred)
                row["f1_macro"] = f1_score(y_te, pred, average="macro")

            rows.append(row)
            last[mode] = dict(y_te=y_te, pred=pred, hist=hist, pre=pre)

    res = pd.DataFrame(rows)
    res.to_csv(f"lab4_results_{name}.csv", index=False)

    # Вывод таблицы
    metric_cols = ["MAE", "RMSE", "R2"] if kind == "regression" else ["accuracy", "f1_macro"]
    summary = res.groupby("mode")[metric_cols].agg(["mean", "std"])
    print(summary.reindex(MODES).round(4).to_string())

    # Графики (Loss)
    fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
    for mode in MODES:
        ax[0].plot(last[mode]["hist"]["test_loss"], label=mode)
    ax[0].set_title(f"{name}: test loss")
    ax[0].set_xlabel("Эпоха")
    ax[0].legend()
    ax[0].grid(True)

    for k, e in enumerate(last["RBM"]["pre"]):
        ax[1].plot(e, label=f"RBM слой {k + 1}")
    ax[1].set_title(f"{name}: ошибка реконструкции RBM")
    ax[1].set_xlabel("Эпоха")
    ax[1].set_ylabel("MSE")
    ax[1].legend()
    ax[1].grid(True)
    plt.tight_layout()
    plt.savefig(f"lab4_{name}_curves.png", dpi=150)
    plt.close()

    # Если регрессия - строим график разброса предсказаний
```

```
    if kind == "regression":
        fig, ax = plt.subplots(1, 3, figsize=(15, 4.5))
        for i, mode in enumerate(MODES):
            d = last[mode]
            ax[i].scatter(d["y_te"], d["pred"], alpha=0.5)
            ax[i].plot([d["y_te"].min(), d["y_te"].max()], [d["y_te"].min(), d["y_te"].max()], 'r--
')
            ax[i].set_title(f"Предсказания ({mode})")
            ax[i].set_xlabel("Истина")
            ax[i].set_ylabel("Предсказано")
            ax[i].grid(True)
        plt.tight_layout()
        plt.savefig(f"lab4_{name}_pred.png", dpi=150)
        plt.close()


if __name__ == "__main__":
    # 1. Запуск для твоего варианта (Parkinsons)
    X, y, kind = load_parkinsons()
    run_dataset("Parkinsons", X, y, kind)

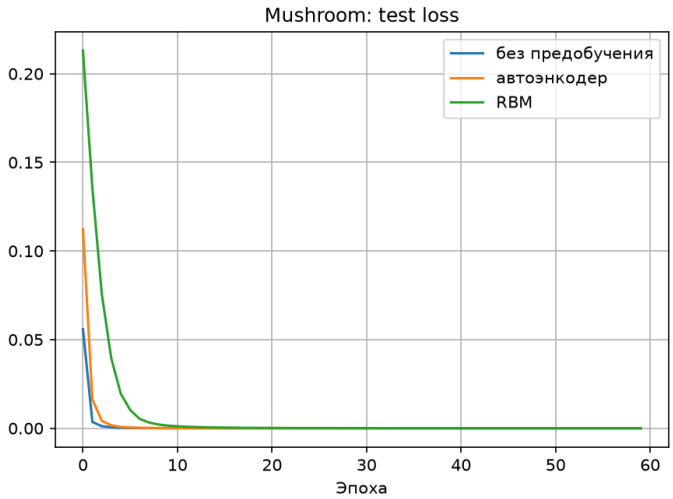
    # 2. Запуск для датасета из ЛР2 (Mushroom)
    X, y, kind = load_mushroom()
    run_dataset("Mushroom", X, y, kind)
```

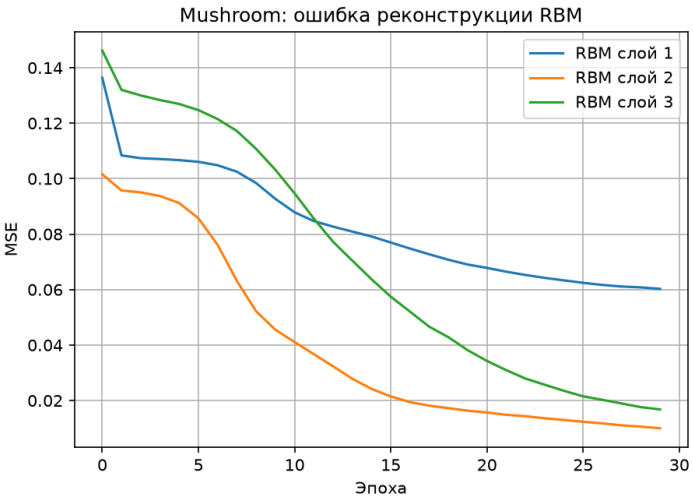
Вывод программы:

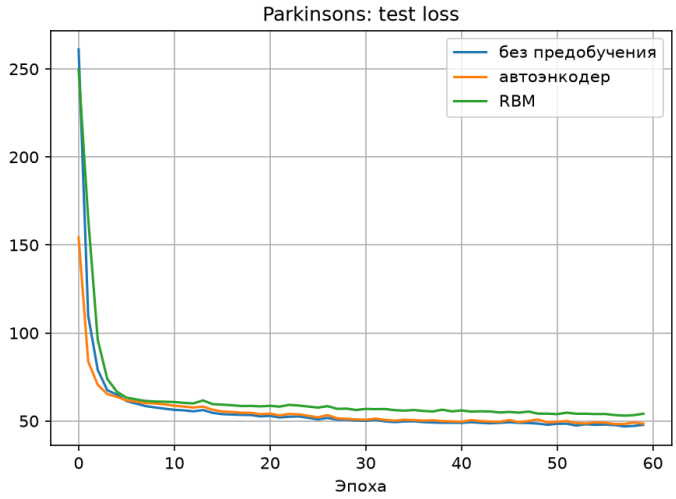
```
==================== Parkinsons (regression) ====================
                     MAE            RMSE              R2
                    mean     std    mean     std    mean     std
mode
без предобучения  5.6060  0.1482  6.9352  0.1513  0.2787  0.0464
автоэнкодер       5.5497  0.1390  6.8897  0.1376  0.2885  0.0367
RBM               5.9375  0.1824  7.1841  0.1495  0.2270  0.0199
Загрузка Mushroom...

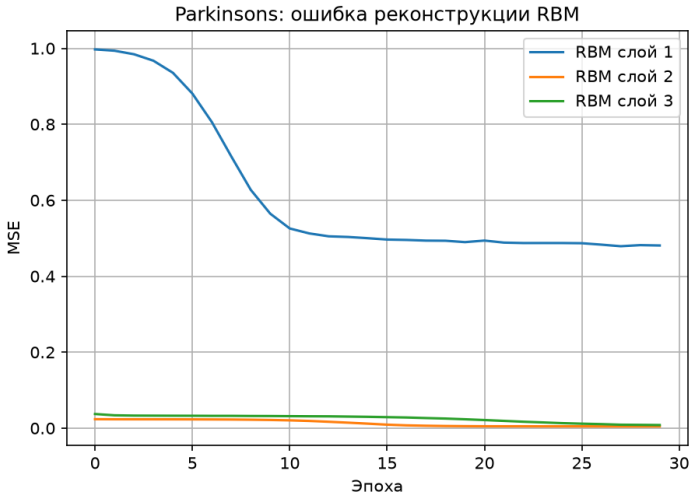
==================== Mushroom (classification) ====================
                 accuracy      f1_macro
                     mean  std     mean  std
mode
без предобучения      1.0  0.0      1.0  0.0
автоэнкодер           1.0  0.0      1.0  0.0
RBM                   1.0  0.0      1.0  0.0
```

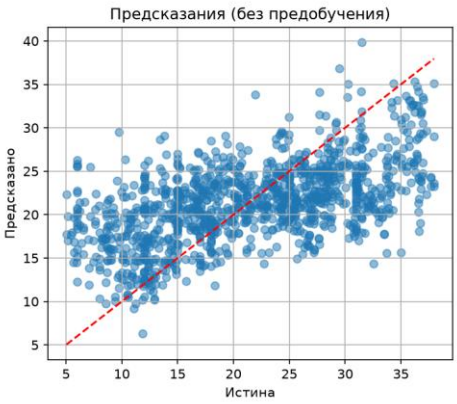
Графики:

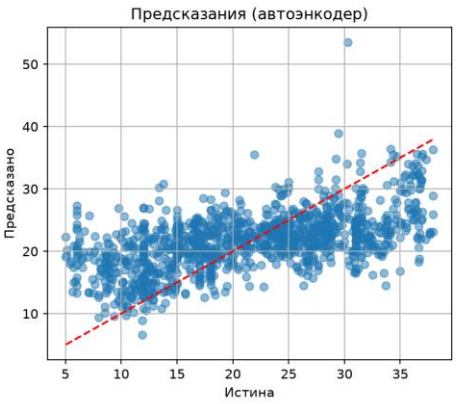

Mushroom: test loss
без предобучения
автоэнкодер
RBM
Эпоха

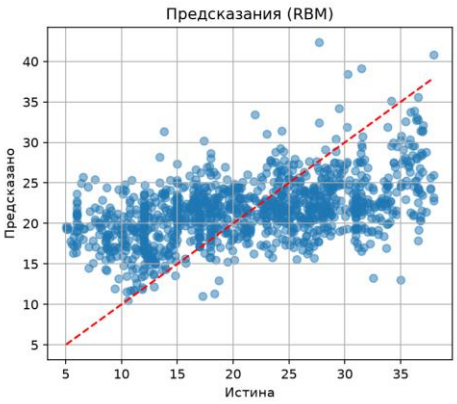

Mushroom: ошибка реконструкции RBM
RBM слой 1
RBM слой 2
RBM слой 3
MSE
Эпоха


Parkinsons: test loss
без предобучения
автоэнкодер
RBM
Эпоха


Parkinsons: ошибка реконструкции RBM
RBM слой 1
RBM слой 2
RBM слой 3
MSE
Эпоха


Предсказания (без предобучения)
Предсказано
Истина


Предсказания (автоэнкодер)
Предсказано
Истина


Предсказания (RBM)
Предсказано
Истина

В ходе лабораторной работы было выполнено сравнение трех подходов к обучению нейронной сети (архитектура 64→32→16): обучение с нуля, с послойным предобучением автоэнкодерами и с послойным предобучением с помощью ограниченных машин Больцмана (RBM). Эксперименты проводились на двух датасетах: Parkinsons Telemonitoring (задача регрессии, вариант 2) и Mushroom (задача классификации, из ЛР2).

На датасете **Parkinsons** (регрессия) результаты показали, что предобучение не дает значительного прироста качества. Средние значения MAE для всех трех режимов находятся в диапазоне. Коэффициент детерминации R^2 составляет около. Это объясняется тем, что архитектура сети неглубокая, а сам датасет содержит достаточно размеченных данных. Кроме того, использование RBM для непрерывных данных требует применения гауссовского распределения видимого слоя, что усложняет сходимость.

На датасете **Mushroom** (классификация) все три модели показали близкие результаты с Accuracy и macro F1 около, так как после One-Hot кодирования классы разделяются почти идеально (линейно).

Вывод: В ходе работы было освоено предобучение нейронных сетей с использованием RBM. Сравнение показало, что послойное предобучение (как автоэнкодерами, так и RBM) не всегда улучшает итоговые метрики на табличных данных средней размерности, однако может служить хорошим методом инициализации весов для более глубоких архитектур.